DLNA Gateway — Architecture (2.0 · ASGI/Hypercorn)

What runs where · stream colours show direction/scope · node codes (P/T/D/E/J) index into the lists overleaf

HTTPS:8443 / HTTP:8765
(Tailscale / LAN)

CLIENTS (inbound)

P/c1 PWA front-end
static/: index.html, app.js,
app.css, sw.js, manifest.json
Service Worker caches:
APP / ART / API (SWR)
MediaSession, offline shell

P/c2 Subsonic client
Amperfy / substreamer /
play:Sub (3rd-party iOS app)
→ CarPlay over Tailscale
talks /rest/* to the gateway,
not to the music server

GATEWAY PROCESS (Hypercorn serves P/g27 dlna_asgi:app; P/g1 start_background_services spawns daemon th

HTTP edge (Hypercorn ASGI · TLS+HTTP/2 :8443 · plain :8765)
P/g27 dlna_asgi (FastAPI) P/g28 dlna_asgi_bridge (legacy shim) P/g30 dlna_events (SSE /api/events)

API handlers (/api/* /rest/* /gw/*)
P/g20 api_browse P/g21 api_playback P/g22 api_playlists
P/g23 api_upnp (gateway-as-MediaServer) P/g24 api_subsonic P/g25 api_radio

Playback & control
P/g15 dlna_player (RendererQueue / QUEUES per UDN)
P/g14 dlna_avtransport (AVTransport+RenderingControl SOAP) P/g16 dlna_stream_proxy (/stream)

AVTransport SOAP
(control · volume)

Discovery
P/g4 dlna_discovery (SSDP, heartbeat, subnet scan) P/g5 dlna_registry P/g6 dlna_devices

Library & index
P/g7 dlna_library (LibraryDB: tracks/FTS5/playlists/overrides…)
P/g8 db_pool (SQLite WAL pool) P/g9 dlna_indexer (crawler)
P/g10 dlna_providers/ (seam: upnp, localfs, mock) P/g13 dlna_content (ContentDirectory SOAP)
DATA → SQLite library.db · config.json · gateway.log

T/* write library.db

ContentDirectory SOAP

read / serve

LocalFs serving (RoHaLocalFS)
P/g11 dlna_localfs_server (bit-perfect file server :8200,
Range/206, DLNA headers, /localfs/stream + /localfs/art) P/g12 dlna_localfs_wiring

HTTP Range → audio bytes
:8200 (bit-perfect)

Background fetchers (event-driven, TLS out)
P/g17 dlna_art_fetcher (MusicBrainz + Cover Art Archive)
P/g29 dlna_art_cache (on-disk cover bytes)
P/g19 dlna_lyrics (lrclib) · P/g26 dlna_config (logging/config)

TLS lookups →

LAN DEVICES (direct, HTTP/SOAP — no TLS)

D/1 Naim Uniti (renderer)
UPnP MediaRenderer
← AVTransport/RenderingControl
SOAP (SetURI/Play/Vol/poll)
→ pulls audio BYTES from :8200
also browses gateway playlists
(UPnP control point)

D/2 UPnP MediaServer
MinimServer / generic UPnP
(AssetUPnP — decommissioned)
via P/g10 UpnpProvider +
P/g13 ContentDirectory SOAP
kept for any non-LocalFs source

D/3 Music files
/Volumes/SAMDATA/Music
(external drive, read-only)
indexed by P/g9, tags read by
P/g10 localfs, bytes served by
P/g11 localfs_server

D/4 fpcalc (local binary)
Chromaprint CLI (brew)
fingerprints files for beets (T/a13)
(local process — internal)

EXTERNAL (TLS out)

E/x1 musicbrainz.org
release-group MBID, year

E/x2 coverartarchive
front-cover presence

E/x3 lrclib.net
on-demand lyrics

E/x4 radio-browser
station catalogue

E/x5 api.acoustid.org
beets tool only (T/a13)

MAINTENANCE TOOLS (tools/*.py — operate directly on library.db / music root)

- T/a1 regen_schema.py
- T/a2 prune_empty_music_dirs.py
- T/a3 find_corrupt_audio.py
- T/a4 find_duplicate_audio.py
- T/a6 relink_orphan_overrides.py
- T/a7 relink_playlists_to_localfs.py
- T/a8 audit_override_mismatches.py
- T/a9 correct_year_drift.py
- T/a10 improve_song_years.py
- T/a11 localfs_scan.py
- T/a12 localfs_serve.py
- T/a13 beets_enrich.py
- T/a14 post_beets_reindex.py

SCHEDULED JOBS & SCRIPTS (launchd)

J/1 com.roha.dlna-gateway
runs the gateway (launchd)

J/2 cert-renew + renew-cert.sh
weekly TLS cert (Mon 04:30)

J/4 setup.sh
venv + run / restart / probe

LEGEND — stream colours

- FROM — inbound client → gateway
- TO — gateway ↔ LAN device
- INTERNAL — within gateway / DB /
- EXTERNAL — gateway → internet (TLS)

Node codes

P = program / module
T = maintenance tool
D = LAN device
E = external service
J = scheduled job / script

Dashed = optional /
intermittent path

Node lists — every box in the drawing

Codes match the diagram. **P** = program / module, **T** = tool, **D** = device, **E** = external service, **J** = scheduled job.

Application programs & modules (P/*)

Code	File / location	Responsibility
P/c1	static/index.html, app.js, app.css, sw.js, manifest.json	PWA front-end. Browse/playback UI, MediaSession lock-screen, Service-Worker caches (APP shell SWR, ART cache-first, API stale-while-revalidate), offline shell.
P/c2	(3rd-party iOS app — Amperfy / substreamer / play:Sub)	Subsonic client for CarPlay over Tailscale. Talks /rest/* to the gateway; never to the music server directly.
P/g1	dlna_gateway.py	Module-wiring + start_background_services(): spawns the daemon threads (SSDP listener, pre-prober, subnet scanner, heartbeat, gateway announcer). Called from the dlna_asgi lifespan so `hypercorn dlna_asgi:app` boots the whole gateway. Its own stdlib HTTP edge + TLS were removed in 2.0 (Hypercorn owns the edge).
P/g3	dlna_routes.py	GET_ROUTES / POST_ROUTES path→handler maps; the ASGI bridge (P/g28) mounts the not-yet-native ones.
P/g4	dlna_discovery.py	SSDP multicast listener, probe, subnet scanner, renderer/server heartbeat. Holds SERVERS / RENDERERS singletons.
P/g5	dlna_registry.py	Data classes + thread-safe ServerRegistry / RendererRegistry.
P/g6	dlna_devices.py	DeviceRoleCache — in-memory mirror of device_roles for zero-latency classification.
P/g7	dlna_library.py	LibraryDB — SQLite index, FTS5 search, playlists, album_art, play_counts, lyrics, metadata_overrides, favourites, radio. Composition root for DB-owning singletons + fetchers.
P/g8	db_pool.py	SQLite connection pool — WAL, thread-local conns, write serialization.
P/g9	dlna_indexer.py	Background crawler that walks a provider and populates LibraryDB (with FTS self-heal).
P/g10	dlna_providers/ (__init__, upnp.py, localfs.py, mock.py)	LibraryProvider seam + registry. UpnpProvider wraps the UPnP SOAP path (kept for MinimServer); LocalFsProvider is the in-process backend (mutagen, content-hashed ids, watchdog).
P/g11	dlna_localfs_server.py	RoHaLocalFS bit-perfect file server (:8200). Range-aware (206/416), DLNA-headered, /localfs/stream/ + /localfs/art/. Path-traversal defended.
P/g12	dlna_localfs_wiring.py	Boot-time wiring of the LocalFs provider (maybe_start_localfs); gated on LOCALFS_MUSIC_ROOT / config.
P/g13	dlna_content.py	UPnP ContentDirectory SOAP client (cd_browse/cd_search). Reached only via P/g10 upnp.py.
P/g14	dlna_avtransport.py	UPnP AVTransport + RenderingControl SOAP (SetURI/Play/Stop/Pause, state/position probe, SetVolume).
P/g15	dlna_player.py	RendererQueue (sequential per-renderer playback, gapless via SetNextAVTransportURI, watchdog) + QueueRegistry (QUEUES, one per UDN).
P/g16	dlna_stream_proxy.py	Browser-audio HTTP proxy /stream (Range pass-through, 5-min idle timeout) + proxy_radio_stream /radio_stream (ICY de-interleave).
P/g17	dlna_art_fetcher.py	AlbumArtFetcher — MusicBrainz release-group + Cover Art Archive lookup; sticky notfound cache; ~1 req/s.
P/g19	dlna_lyrics.py	On-demand lrclib lyrics; cached in the lyrics table; sticky positive + negative.
P/g20	api_browse.py	Browse / search / radio-shuffle API.
P/g21	api_playback.py	Playback, /stream route, /art proxy, /api/client_log, state, indexer + AcoustID management.
P/g22	api_playlists.py	Playlist CRUD endpoints.
P/g23	api_upnp.py	UPnP service descriptors + ContentDirectory SOAP exposing the gateway's own playlists + favourites (so the Naim can browse it as a MediaServer).
P/g24	api_subsonic.py	Subsonic-compatible /rest/* API (browse, playlists, favourites→starred, stream, cover, scrobble, internet radio) for CarPlay clients.
P/g25	api_radio.py	Internet-radio endpoints: radio-browser search, favourites (≤25), ICY now-playing.
P/g26	dlna_config.py	Constants (DB_FILE/CFG_FILE/LOG_FILE), logging setup, config load/save, .env load (if dotenv present), raise_fd_limit(8192).
P/g27	dlna_asgi.py	2.0 — THE server (Hypercorn owns the whole edge). FastAPI app: TLS+HTTP/2 on :8443 via ALPN, plain :8765; owns the tailscale cert. Native routes for the read API, /art, /stream + /radio_stream relays, static/PWA, the Subsonic byte methods, and the Naim-facing /gw/* UPnP surface (device.xml/desc.xml/events/control on the plain :8765 bind — Cleanup C folded it in here, retiring the separate device server). Remaining legacy handlers run via the bridge. Lifespan boots P/g1 services. docs_url off (no CDN call).
P/g28	dlna_asgi_bridge.py	Shim that runs the legacy (h, params) handlers unchanged inside the ASGI app (a fake `h` captures _json/_html/_xml/send_error; runs in a threadpool). Routes are rewritten native one batch at a time, then dropped from the bridge.
P/g29	dlna_art_cache.py	On-disk cover-art byte cache keyed by source URL. art_fetch_cached() (in P/g21) fronts art_fetch so /art and Subsonic getCoverArt serve repeat covers from disk (across clients + restarts) instead of re-fetching coverartarchive / re-decoding embedded art. TTL + size capped; art_cache/ gitignored.
P/g30	dlna_events.py	EventBus/EVENTS — thread-safe publish to the asyncio loop; native GET /api/events (SSE). Publishers: RendererQueue state, index status transitions, discovery changes. The PWA opens an EventSource as a polling accelerator (fallback intact).

Maintenance tools (T/*) — tools/*.py

Operate directly on library.db and/or the music root. All default to dry-run / report-only; deletions move to Trash unless --hard-delete. Each has a unit-test sibling tools/test_*.py.

Code	File	Purpose & key options
T/a1	regen_schema.py	Regenerate the committed schema.sql artifact after any schema change. **--check** = fail if stale (the test_schema_sync gate).
T/a2	prune_empty_music_dirs.py	Trash directories whose subtree has zero music files. **--dry-run -v --limit N --exts a,b --hard-delete -y**.
T/a3	find_corrupt_audio.py	Flag files with bad/zero magic bytes for their extension. Writes corrupt-audio.txt. **--trash --hard-delete --limit --exts --out -y**.
T/a4	find_duplicate_audio.py	Find duplicate recordings on disk (same acoustid metadata); ranks a winner by bit-depth/sample-rate/size. **--trash --hard-delete -v -y**.
T/a6	relink_orphan_overrides.py	Relink metadata_overrides orphaned by a UPnP rescan via d-id + fuzzy (artist,title). **--apply --db**.
T/a7	relink_playlists_to_localfs.py	Repoint playlist_tracks/favourites at RoHaLocalFS by normalised metadata; prune no-match. **--apply --no-backup --no-prune-favs -y**.
T/a8	audit_override_mismatches.py	Delete acoustid overrides whose artist AND title both mismatch the track (d-id collision repair). **--clean --top -y**.
T/a9	correct_year_drift.py	Rewrite metadata_overrides.year to the earliest plausible year evidenced by another copy in the library. **--apply --top --db -y**.
T/a10	improve_song_years.py	Query MusicBrainz for each song's earliest recording year → song_year_cache → apply. **--lookup --apply --limit -v**.
T/a11	localfs_scan.py	Standalone LocalFs indexer (CLI). NOTE: do not run against the live DB without a base_url — leaves placeholder URLs.
T/a12	localfs_serve.py	Standalone LocalFs file-server launcher for testing the :8200 serving path.
T/a13	beets_enrich.py	Run the beets tag-in-place enrichment batch (docs/enrichment.md). Safe wrapper around `beet import`: enforces write:yes/copy:no/move:no before any write. **--write-config --quiet --timid --album --revisit --reindex --gateway --dry-run -y**.
T/a14	post_beets_reindex.py	The post-beets step: drop AcoustID metadata_overrides (LocalFs URLs are path-stable, so old acoustid rows would re-mask beets' fresh tags) then reindex LocalFs. Refuses to clear while ACOUSTID_API_KEY is set. **--apply --dry-run --no-clean --no-reindex --no-backup --ignore-acoustid-key --udn --gateway --db -y**.

LAN devices (D/*), external services (E/*), scheduled jobs (J/*)

Code	Name	Role / protocol
D/1	Naim Uniti	UPnP MediaRenderer. Receives AVTransport+RenderingControl SOAP (control/volume); pulls audio bytes over HTTP Range from :8200; also browses gateway playlists as a control point. Gateway is NOT in its audio path → bit-perfect.
D/2	UPnP MediaServer	MinimServer / generic UPnP (AssetUPnP decommissioned). Browsed via P/g10 UpnpProvider + P/g13 ContentDirectory SOAP. Kept for any non-LocalFs source.
D/3	Music files	/Volumes/SAMDATA/Music (external drive, read-only). Indexed by P/g9; tags read by localfs provider; bytes served by P/g11.
D/4	fpcalc	Chromaprint CLI binary (brew). Local fingerprinter for the beets tool (T/a13). Internal/local — not network.
E/x1	musicbrainz.org	GET /ws/2/release-group — MBID + original year. UA + 1.1 s rate limit required.
E/x2	coverartarchive.org	HEAD /release-group/{mbid}/front-500 — cover presence.
E/x3	lrclib.net	GET /api/get — on-demand lyrics for the playing track.
E/x4	*.api.radio-browser.info	GET /json/stations/search — internet-radio catalogue (HLS filtered).
E/x5	api.acoustid.org	fingerprint → MusicBrainz metadata — used ONLY by the beets tool (T/a13) now; the in-process AcoustID worker was removed in 2.0.
J/1	com.roha.dlna-gateway	LaunchAgent that runs the gateway. Restart: launchctl kickstart -k gui/$(id -u)/com.roha.dlna-gateway.
J/2	com.roha.dlna-cert-renew + renew-cert.sh	Weekly TLS cert renewal (Mon 04:30; no-op unless <30 days). cert-renewal.log.
J/4	setup.sh	venv setup + run / restart / probe. --run --restart --no-browser --debug --probe URL --list-devices --reset-devices. See the setup.sh options reference.

Commands & options used in the drawing

Context	Command
Run (J/4)	**./setup.sh --run [--no-browser] [--debug] [--probe http://…] [--list-devices] [--reset-devices]**
Restart (J/4)	**./setup.sh --restart (refresh venv/deps + launchctl kickstart -k gui/$(id -u)/com.roha.dlna-gateway)**
Cert (J/2)	**./renew-cert.sh [--force] · launchctl kickstart gui/$(id -u)/com.roha.dlna-cert-renew**
Subsonic auth	**launchctl setenv SUBSONIC_PASSWORD … ; launchctl getenv SUBSONIC_PASSWORD**
Full test suite	**python tests/run_all.py [--offline | --frontend | --frontend-only | http://host:8765]**
Unit / Playwright	**python3 -m unittest tests.test_player … ; .venv/bin/pytest tests/frontend -v**
Chaos (live)	**python3 tests/chaos.py --iterations N --workers M [--seed S] [--quiet] [--base https://host:8443]**
Schema gate (T/a1)	**python3 tools/regen_schema.py [--check]**
Module self-tests	**python dlna_discovery.py | dlna_content.py | dlna_library.py | db_pool.py | dlna_player.py**
Range / 206 check	**curl -r 0-1023 -D - http://<host>:8200/localfs/stream/<id> -o /dev/null**
Tools (T/a2–a12)	**python3 tools/<tool>.py [--dry-run|--apply|--trash|--hard-delete|--limit N|--exts a,b|-v|-y] (see the tools table above)**

Ports (2.0): 8443 HTTPS — Hypercorn TLS + HTTP/2 (ALPN), tailscale cert · 8765 plain HTTP · 8200 RoHaLocalFS file server · 8770 device-tier /gw/* (plain, for the Naim; SSDP advert points here) · 26125 (legacy AssetUPnP, decommissioned). The HTTP/2 + app-owned-TLS roadmap is now DONE — Hypercorn terminates TLS/h2 natively. Cutover: docs/CUTOVER_RUNBOOK.md + CUTOVER_LAUNCHD.md.

Tool options reference — what each flag does

Every option for every maintenance tool, with its meaning. Codes match the diagram (T/aN) and the tools table. Unless noted, file-deleting tools default to dry-run / report-only and move to the macOS Trash (recoverable ~30 days) — **--hard-delete** is the permanent, non-recoverable variant.

T/a1 regen_schema.py — Regenerate the committed schema.sql.

Option	Meaning
(no args)	Rewrite schema.sql from the live LibraryDB schema.
--check	Don't write; exit non-zero if schema.sql is stale (the test_schema_sync gate).

T/a2 prune_empty_music_dirs.py — Trash directories whose subtree has zero music files.

Option	Meaning
	Music root to walk (positional).
--dry-run	Print decisions; touch nothing.
-v / --verbose	Also log every KEPT directory (default: only deletions print).
--limit N	Stop after evaluating N dirs; when the limit is hit, NO deletions run (a partial picture = safety belt).
--exts a,b,c	Override the music-extension set (commas, dot optional).
--hard-delete	Permanent rm -rf instead of Trash. NOT recoverable.
-y / --yes	Skip the confirmation prompt.

T/a3 find_corrupt_audio.py — Flag files whose magic bytes are wrong/zero for their extension.

Option	Meaning
	Music root to scan (positional).
-v / --verbose	Also log every OK file (default: only corrupt).
--limit N	Stop after scanning N files (0 = no limit); halts the delete step when hit.
--exts a,b,c	Override the audio-extension list.
--out PATH	Where to write the corrupt-paths list (default ./corrupt-audio.txt; pass /dev/null to suppress).
--trash	Move flagged files to the Trash.
--hard-delete	Permanent unlink instead of Trash. Mutually exclusive with --trash.
-y / --yes	Skip the confirmation prompt before deleting.

T/a4 find_duplicate_audio.py — Find duplicate recordings on disk (same acoustid metadata); keeps a ranked winner.

Option	Meaning
	Music root to scan (positional).
-v / --verbose	Per-URL ambiguity / not-found logs.
--trash	Move the loser files to the Trash (winner kept).
--hard-delete	Permanent delete of losers. NOT recoverable.
-y / --yes	Skip the confirmation prompt.

T/a6 relink_orphan_overrides.py — Relink metadata_overrides orphaned by a UPnP rescan (d-id + fuzzy artist/title).

Option	Meaning
(default)	Dry-run preview — no mutation.
--apply	Perform the relinks.
--db PATH	library.db path.

T/a7 relink_playlists_to_localfs.py — Repoint playlists/favourites at RoHaLocalFS by normalised metadata.

Option	Meaning
(default)	Dry-run preview.
--apply	Commit the relinks (auto-backs up library.db first).
--no-backup	Skip the automatic library.db backup on --apply.
--no-prune-favs	Keep album_favourites that no longer match any LocalFs album.
-y / --yes	Skip the confirmation prompt.

T/a8 audit_override_mismatches.py — Delete acoustid overrides whose artist AND title both mismatch the track.

Option	Meaning
(default)	Dry-run, lists the top 30 suspects.
--clean	Delete the suspect acoustid override rows.
--top N	Show N rows (0 = full list).
-y / --yes	Non-interactive (skip the confirm).

T/a9 correct_year_drift.py — Rewrite metadata_overrides.year to the earliest plausible year from another copy in the library.

Option	Meaning
(default)	Dry-run, top 30 candidates.
--apply	Write the corrections (as source='manual').
--top N	Preview N candidates (0 = all).
--db PATH	library.db path.
-y / --yes	Non-interactive apply.

T/a10 improve_song_years.py — Query MusicBrainz for each song's earliest recording year → cache → apply.

Option	Meaning
(default)	Dry-run preview — no MB calls, no DB writes.
--lookup	Query MB for uncached (artist,title) groups; fill song_year_cache (sticky).
--apply	Write cached hits onto tracks (metadata_overrides.year, source='manual').
--limit N	Cap the number of groups queried (for testing).
-v / --verbose	Per-group logging.

T/a11 localfs_scan.py — Standalone LocalFs indexer (CLI). Don't run against the live DB without a base_url — it leaves placeholder URLs.

Option	Meaning
--root PATH	Music root to scan (default $LOCALFS_MUSIC_ROOT or /Volumes/SAMDATA/Music).
--force	Ignore the (mtime, size) cache and re-tag every file. Use after schema changes.
--compare	Skip scanning — just print tracks/albums per UDN currently in library.db.
--db PATH	library.db path.
-v / --verbose	Per-file logging.

T/a12 localfs_serve.py — Standalone RoHaLocalFS file-server launcher (test the :8200 path).

Option	Meaning
--port N	HTTP port (default 8200 / $LOCALFS_PORT).
--host ADDR	Listen address (default 0.0.0.0 so the Naim can reach it).
--db PATH	library.db path.
--root PATH	Allowed music root; repeatable. Defaults to $LOCALFS_MUSIC_ROOT.
-v / --verbose	Request logging.

T/a13 beets_enrich.py — Run the beets tag-in-place enrichment batch (docs/enrichment.md). Safe wrapper around `beet import`. Deps: brew install chromaprint; pip3 install beets pyacoustid musicbrainzngs (beets 2.x pluginized MusicBrainz — the musicbrainz plugin + musicbrainzngs are REQUIRED or it matches nothing).

Option	Meaning
(no --quiet/--timid)	Interactive: beets prompts you per album (apply / skip / …); strong matches auto-apply.
--write-config	Write the prog-tuned tag-in-place ~/.config/beets/config.yaml (enables the musicbrainz metadata plugin; backs up any existing) and exit. Run this first.
--music-root PATH	Library root to import (default /Volumes/SAMDATA/Music).
--album PATH	Import a single album directory instead of the whole root.
--config PATH	beets config path (default ~/.config/beets/config.yaml).
--quiet	Unattended BULK pass: auto-accept only strong matches (≥ strong_rec_thresh 0.80), skip the rest, no prompts (beet import -q).
--timid	Most cautious: prompt for EVERY match (per-change review). Mutually exclusive with --quiet.
--revisit	Re-import a directory beets already recorded as done (-I / noincremental). Use after re-tagging an album.
--reindex	After import, find the LocalFs server UDN via /api/servers and POST /api/index/rebuild so the gateway picks up the new tags.
--gateway URL	Gateway base URL for --reindex (default http://127.0.0.1:8765).
--udn UDN	Explicit server UDN to reindex (default: auto-pick the uuid:localfs-* server).
-n / --dry-run	Print the resolved beet command + safety report; do NOT invoke beets (beets has no true dry-run of its own).
-y / --yes	Skip the in-place-write backup-warning confirmation.

T/a14 post_beets_reindex.py — After beets has tagged files in place, make its work visible: clear the AcoustID metadata_overrides (LocalFs URLs are PATH-stable, so the COALESCE pass would otherwise re-lay old acoustid rows over beets' fresh tags) THEN reindex LocalFs. source='manual' is never touched; notfound/video_skip carry NULL metadata so they mask nothing.

Option	Meaning
(default)	Dry-run: print the override breakdown + the planned clear/reindex; change nothing.
--apply	Actually delete acoustid overrides + start the reindex (auto-backs up library.db first).
-n / --dry-run	Explicit preview alias (the default when --apply is absent).
--no-clean	Skip clearing overrides (reindex only).
--no-reindex	Skip the reindex (clean only).
--no-backup	Skip the library.db backup before deleting.
--ignore-acoustid-key	Proceed even if ACOUSTID_API_KEY is set. By default the tool REFUSES to clear while the worker is live — the 120s startup scan would re-fingerprint every now-bare track and re-create the overrides, re-masking beets (Option A keeps the key unset).
--udn UDN	Server UDN to reindex (default: auto-pick uuid:localfs-*).
--gateway URL	Gateway base URL (default http://127.0.0.1:8765).
--db PATH	library.db path.
-y / --yes	Skip the confirmation prompt.

J/4 setup.sh — Bootstrap + run the gateway. Finds Python 3.9+, creates/repairs the .venv, installs requirements.txt, then either runs the gateway (--run, exec in the foreground), restarts the launchd-managed copy (--restart), or just finishes setup. Unknown flags after --run are forwarded verbatim to dlna_gateway.py.

Option	Meaning
(no flags)	Set up only: venv + dependencies, then print the 'how to run' summary. Does not start anything.
--run	Set up, then exec the gateway in the foreground on :8765 (blocks; Ctrl-C to stop). All flags below it are passed through to dlna_gateway.py.
--restart	Refresh the venv/deps, then restart the launchd gateway via launchctl kickstart -k gui/$(id -u)/com.roha.dlna-gateway (the launchd-correct restart — a bare kill races launchd's respawn). Aborts with install hints if the LaunchAgent isn't loaded. Mutually informative with --run; --restart wins if both are given.
--no-browser	(forwarded) Don't auto-open the browser on start.
--debug	(forwarded) Verbose logging.
--probe http://…	(forwarded) Add a UPnP server manually by its device-description URL.
--list-devices	(forwarded) Print the known-devices table and exit.
--reset-devices	(forwarded) Wipe the device DB and exit.
--port N	(forwarded) HTTP port (default 8765).